

Guía Completa de Positron IDE

Edison Achalma

Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Positron es un IDE moderno y especializado en ciencia de datos, construido sobre el núcleo open-source de VS Code (Code OSS) y desarrollado por Posit Software. Ofrece soporte nativo para Python y R, consola interactiva avanzada, paneles especializados (Variables, Data Explorer, Plots, Help), integración completa con Quarto y herramientas de asistencia IA. Esta guía completa cubre desde la instalación multiplataforma hasta la configuración avanzada, migración desde VS Code, personalización profunda y mejores prácticas para flujos de trabajo intensivos en análisis de datos, programación estadística y publicación científica reproducible.

Palabras Claves: Positron IDE, Ciencia de datos, Python, R, Quarto

Tabla de contenidos

Introduction	4
1 Guía Completa de Positron: Instalación, Configuración y Personalización	4
1.1 Índice	4
1.2 1. ¿Qué es Positron?	5
1.2.1 Positron vs. VS Code vs. RStudio	5
1.3 2. Requisitos previos	6
1.3.1 Python	6
1.3.2 R (opcional, si lo vas a usar)	6
1.4 3. Instalación en Linux	7
1.4.1 Paso 1 — Descargar el instalador	7
1.4.2 Paso 2 — Instalar el paquete .deb	7
1.4.3 Paso 3 — Verificar la instalación	7
1.4.4 Alternativa: Arch Linux / Archcraft	7
1.4.5 Notas importantes para Linux	8

Edison Achalma  <https://orcid.org/0000-0001-6996-3364>

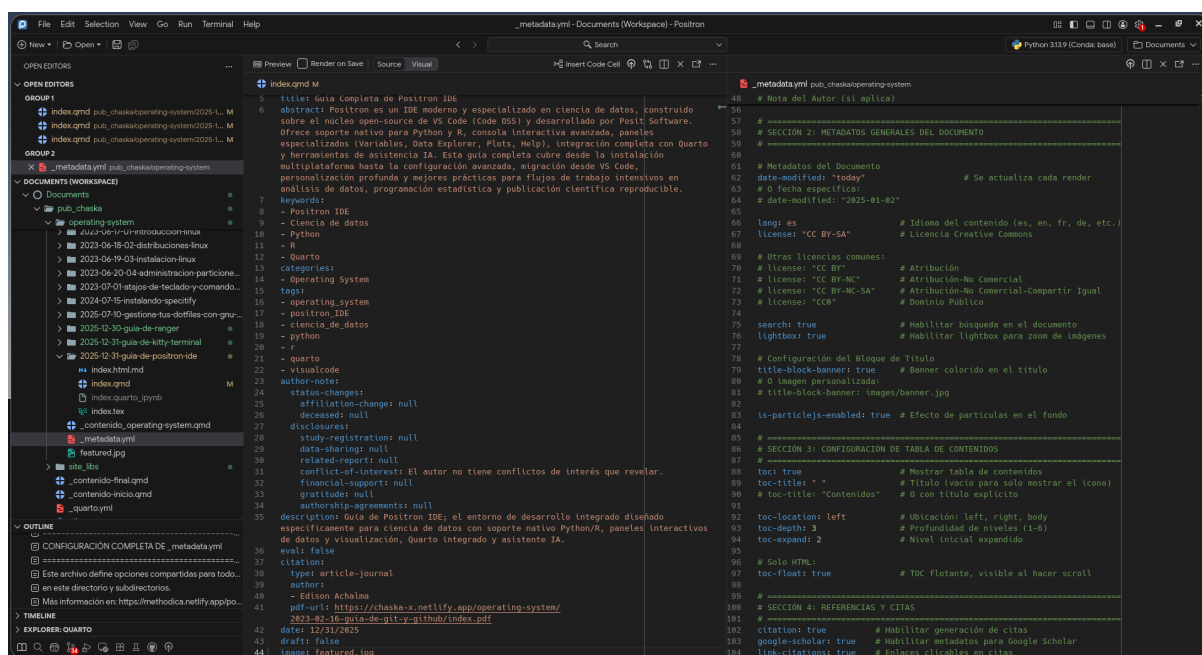
El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; <https://credit.niso.org/>) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

1.5	4. Primer arranque y comando en terminal	8
1.5.1	Abrir Positron	8
1.5.2	Importar configuración desde VS Code	8
1.6	5. Actualizaciones	9
1.6.1	Desactivar la verificación automática de actualizaciones	9
1.6.2	Canal de <i>daily builds</i> (opcional, para probar features nuevas)	9
1.7	6. Tour de la interfaz	10
1.7.1	Layout general (heredado de Code OSS)	10
1.7.2	Paneles exclusivos de Positron	10
1.7.3	Console vs. Terminal — la distinción clave	10
1.8	7. Atajos de teclado esenciales	11
1.8.1	Atajos específicos de Positron (los más importantes)	11
1.8.2	Atajos generales heredados de VS Code	11
1.8.3	Crear tus propios atajos	12
1.9	8. Extensiones: Open VSX y P3M	12
1.9.1	Por qué Positron no usa el VS Code Marketplace	12
1.9.2	Extensiones incluidas por defecto	13
1.9.3	Extensiones NO compatibles — evitar instalarlas	13
1.9.4	Confianza y seguridad de extensiones	13
1.9.5	Extensiones recomendadas para instalar manualmente	14
1.9.6	Instalación manual (offline / .vsix)	14
1.10	9. Dónde vive la configuración	14
1.10.1	Ubicación del archivo <code>settings.json</code>	14
1.10.2	Cómo abrirlo	14
1.10.3	Settings por workspace (proyecto específico)	14
1.11	10. Personalización visual (temas, iconos, fuentes)	15
1.11.1	Tema de color	15
1.11.2	Tema de iconos de archivos	15
1.11.3	Personalizar colores específicos	15
1.11.4	Tipografía	15
1.12	11. Personalización del layout	16
1.12.1	Posición de la barra de actividad (íconos de Explorer, Git, etc.)	16
1.12.2	Posición de la barra lateral principal	16
1.12.3	Pestañas de editor	16
1.12.4	Barra de estado y breadcrumbs	16
1.12.5	Modo Zen (máxima concentración, sin distracciones)	16
1.13	12. Configuración del editor	16
1.13.1	Lo esencial	16
1.13.2	Configuración por lenguaje (muy útil si trabajas Python + Quarto + LaTeX + R)	17
1.13.3	Python: intérprete y kernel	17
1.13.4	R: formateo con Air	17
1.13.5	Quarto	18
1.13.6	Terminal integrada	18
1.14	13. Configuración de Git	18
1.14.1	Configuración inicial de Git (una sola vez, fuera de Positron)	18
1.14.2	Settings recomendados de Git en Positron	18
1.14.3	Panel de Source Control	19

1.15	14. Posit Assistant (IA integrada)	19
1.15.1	Activarlo	19
1.15.2	Usarlo	19
1.15.3	Databot	19
1.16	15. Atajos personalizados (keybindings.json)	19
1.16.1	Ejemplo: abrir el Data Explorer con F2 desde el panel de Variables	19
1.16.2	Ejemplo: generar un reprex en R desde tu selección	20
1.17	16. Solución de problemas comunes	20
1.17.1	El intérprete de Python no se detecta	20
1.17.2	R no funciona	20
1.17.3	La Console no inicia	21
1.17.4	Las extensiones esperadas no aparecen	21
1.17.5	La terminal no hereda el PATH correctamente	21
1.17.6	Los plots no aparecen (Python)	21
1.17.7	Quarto no renderiza	21
1.17.8	Los settings no se guardan	21
1.17.9	Rendimiento lento	21
1.18	17. Mi configuración final (settings.json)	22
2	Publicaciones Similares	26

Guía Completa de Positron IDE



title: “Guía Completa de Positron” subtitle: “Instalación, Configuración y Personalización” author: “Edison Achalma — UNSCH, Ayacucho” fecha: “Junio 2026” version-positron: “2026.06.0”

1 Guía Completa de Positron: Instalación, Configuración y Personalización

Guía de referencia personal para arrancar con Positron desde cero en Linux (Kubuntu / Arch), dejarlo configurado a mi gusto y tener mi `settings.json` final listo para copiar y pegar. Basada en la documentación oficial de Posit (`positron.posit.co`) — versión **Positron 2026.06.0**.

1.1 Índice

1. ¿Qué es Positron?
2. Requisitos previos
3. Instalación en Linux
4. Primer arranque y comando en terminal
5. Actualizaciones
6. Tour de la interfaz
7. Atajos de teclado esenciales
8. Extensiones: Open VSX y P3M
9. Dónde vive la configuración
10. Personalización visual (temas, iconos, fuentes)
11. Personalización del layout
12. Configuración del editor

13. Configuración de Git
14. Posit Assistant (IA integrada)
15. Atajos personalizados (`keybindings.json`)
16. Solución de problemas comunes
17. Mi configuración final (`settings.json`)

1.2 1. ¿Qué es Positron?

Positron es un IDE moderno y extensible para ciencia de datos, construido sobre **Code OSS** (el núcleo open-source de VS Code) y desarrollado por **Posit Software, PBC** (antes RStudio). A diferencia de VS Code, trae soporte **nativo** para Python y R, sin depender de extensiones externas frágiles.

Características principales:

- Basado en Code OSS → hereda toda la infraestructura de VS Code (paleta de comandos, atajos, configuración por `settings.json`, extensiones).
- Soporte nativo de Python y R, con consola interactiva (kernel Ark, IPython/IRKernel).
- **Variables Pane**: inspección en vivo de variables del entorno.
- **Data Explorer**: exploración interactiva de dataframes.
- **Plots Pane**: historial y exportación de gráficos.
- **Help Pane**: documentación contextual integrada.
- **Connections Pane**: gestión de conexiones a bases de datos.
- **Packages Pane**: gestión visual de paquetes R/Python.
- Soporte nativo de **Quarto**, incluyendo salida en línea (*inline output*) para documentos `.qmd`.
- **Posit Assistant**: asistente de IA integrado en el IDE (sucesor del antiguo “Positron Assistant”).
- Galería de extensiones servida desde **P3M** (Posit Public Package Manager), con el mismo catálogo que Open VSX.
- Licencia: **Elastic License 2.0** (source-available, gratuito para uso individual).

1.2.1 Positron vs. VS Code vs. RStudio

Característica	Positron	VS Code	RStudio
Base técnica	Code OSS	Propietario + Code OSS	Propietario (Posit)
Soporte nativo Python	Sí	No (requiere extensión)	Limitado
Soporte nativo R	Sí	No (requiere extensión)	Sí (muy optimizado)
Consola interactiva	Sí	No	Sí
Variables / Data Explorer / Plots	Sí	No	Sí
Soporte Quarto	Sí (nativo)	Sí (extensión)	Sí
Galería de extensiones	P3M / Open VSX	VS Marketplace	Limitada

Característica	Positron	VS Code	RStudio
Asistente IA integrado	Sí (Posit Assistant)	No (requiere Copilot)	No
Multilenguaje (R + Python)	Sí	Sí (muy amplio)	Limitado (foco en R)

Cuándo usar cada uno: si tu trabajo combina R y Python (como flujos con *reticulate*, Quarto multi-lenguaje, etc.), Positron es la opción natural. Si tu trabajo es 100 % R y dependes de funciones muy específicas de RStudio (ciertos *add-ins*, Sweave, publishing tools de nicho), RStudio sigue siendo una alternativa sólida y bien mantenida.

1.3 2. Requisitos previos

Antes de instalar, conviene tener listo lo siguiente (documentación oficial: *Install Positron*):

- **Sistema operativo:** Linux (Ubuntu/Debian o RHEL-based), macOS o Windows.
- **Espacio en disco:** ~500 MB mínimo para el propio IDE.
- **RAM:** 2 GB mínimo, 4 GB recomendado.

1.3.1 Python

- Positron soporta las versiones **activamente mantenidas de Python, de la 3.9 a la 3.13**.
- Gestores de versiones recomendados: **uv** (recomendado por Posit) o **pyenv**.
- Positron requiere el **kernel IPython** (versión 6.19.1 o superior) para comunicarse con Python. Por defecto, este kernel viene **integrado (bundled)** con Positron para la implementación estándar de CPython.
- Si prefieres control fino sobre tu entorno, puedes desactivar el kernel integrado con `python.useBundledIpykernel: false` e instalar `ipykernel` manualmente vía `pip install ipykernel`.

1.3.2 R (opcional, si lo vas a usar)

- Se requiere **R 4.2 o superior**. Instálalo siguiendo las instrucciones de tu sistema en cloud.r-project.org.
- Si quieres manejar múltiples versiones de R en paralelo, la herramienta **rig** funciona muy bien con Positron.
- Si desarrollas paquetes R, conviene tener actualizados estos paquetes (mejoran la integración con Positron):

```
# Con pak (recomendado por Posit)
pak::pak(c("usethis", "cli", "crayon", "rlang", "roxygen2", "pkgload"))

# O con base R
install.packages(c("usethis", "cli", "crayon", "rlang", "roxygen2", "pkgload"))
```

1.4 3. Instalación en Linux

Positron se distribuye oficialmente como paquete `.deb` (Debian/Ubuntu, incluye Kubuntu) y `.rpm` (Red Hat/Fedora), para arquitecturas **x64** y **arm64**.

1.4.1 Paso 1 — Descargar el instalador

Descarga el paquete correspondiente a tu arquitectura desde la página oficial de descargas:

<https://positron.posit.co/download.html>

Para Kubuntu/Ubuntu (x64), el paquete tiene este formato de nombre:

Positron-2026.06.0-211-x64.deb

O directamente por terminal:

```
# Debian/Ubuntu/Kubuntu x64
wget https://cdn.posit.co/positron/releases/deb/x86_64/Positron-2026.06.0-211-x64.deb

# Debian/Ubuntu arm64
wget https://cdn.posit.co/positron/releases/deb/arm64/Positron-2026.06.0-211-arm64.de
```

Verifica siempre la versión más reciente en la página oficial, ya que el número de build cambia con cada release mensual.

1.4.2 Paso 2 — Instalar el paquete `.deb`

```
# Instalar el paquete descargado
sudo dpkg -i Positron-2026.06.0-211-x64.deb

# Resolver dependencias faltantes, si las hay
sudo apt-get install -f
```

1.4.3 Paso 3 — Verificar la instalación

```
positron --version
```

Si el comando no se reconoce, revisa la sección de solución de problemas.

1.4.4 Alternativa: Arch Linux / Archcraft

Positron no se distribuye oficialmente como paquete `.rpm`/`.deb` para Arch, pero puedes:

- Usar un paquete de la **AUR** (por ejemplo `positron-bin`), si está disponible y confías en el mantenedor.
- O extraer el `.deb` manualmente con herramientas como `ar` y `tar` y colocar los binarios en `/opt/positron`, creando luego un symlink (ver sección 4).

```
# Extracción manual de un .deb en un sistema sin dpkg (ej. Arch)
mkdir positron-extract && cd positron-extract
ar x ../Positron-2026.06.0-211-x64.deb
tar -xzf data.tar.xz
# El binario queda típicamente en ./usr/share/positron/
```

1.4.5 Notas importantes para Linux

- El instalador .deb **no se autoactualiza** como en macOS/Windows: en Linux, Positron solo **te notifica** cuando hay una actualización disponible; debes descargar e instalar el nuevo .deb/.rpm manualmente.
- Existe un detalle conocido y ya corregido en versiones recientes: versiones antiguas del desinstalador .deb podían eliminar por error las fuentes APT y la llave de firma de VS Code si lo tenías instalado también. Esto está solucionado desde el changelog de 2026.06.0.

1.5 4. Primer arranque y comando en terminal

1.5.1 Abrir Positron

```
# Desde el lanzador de aplicaciones de tu DE (KDE/otros), o desde terminal:
positron

# Abrir Positron en la carpeta actual
positron .

# Abrir Positron en una carpeta específica
positron /ruta/a/mi/proyecto
```

En Linux, el paquete .deb/.rpm coloca el binario directamente en el PATH del sistema durante la instalación, así que normalmente **no necesitas configurar nada adicional** para usar el comando `positron` desde la terminal (a diferencia de macOS, donde hay que ejecutar “*Install ‘positron’ command in PATH*” desde la paleta de comandos).

Si por alguna razón el comando no está disponible (instalación manual, symlink roto, etc.), créalo tú mismo:

```
# Crear un symlink manual hacia el binario
sudo ln -s /usr/share/positron/positron /usr/local/bin/positron

# O añadir la ruta a tu PATH en ~/.zshrc (Edison usa zsh)
echo 'export PATH="$PATH:/usr/share/positron"' >> ~/.zshrc
source ~/.zshrc
```

1.5.2 Importar configuración desde VS Code

Si vienes de VS Code, en el primer arranque Positron puede ofrecerte importar tu configuración existente:

1. Acepta el prompt de importación al iniciar por primera vez.
2. Revisa los *settings* importados (algunos no aplican porque Positron tiene su propia gestión de Python/R nativa).
3. Guarda los cambios.

También puedes hacerlo manualmente en cualquier momento:

Paleta de Comandos (Ctrl+Shift+P) → Preferences: Import Settings...

Importante: extensiones como `ms-python.python` o `REditorSupport.r` **no deben usarse** en Positron — ya tiene soporte nativo equivalente e incompatibilidades conocidas con ambas (ver sección 8).

1.6 5. Actualizaciones

Comportamiento oficial de actualización por plataforma:

- **macOS** y la instalación de usuario en **Windows**: si hay una actualización, Positron la descarga e instala automáticamente en segundo plano.
- **Windows (instalación de sistema)**: descarga la actualización y te avisa con un ícono en la esquina inferior izquierda.
- **Linux**: Positron **muestra una notificación** cuando hay una actualización disponible, pero la instalación del nuevo paquete `.deb/.rpm` la haces tú manualmente.

1.6.1 Desactivar la verificación automática de actualizaciones

```
{  
  "update.mode": "none"  
}
```

1.6.2 Canal de daily builds (opcional, para probar features nuevas)

Si quieres recibir compilaciones diarias en lugar de la release mensual estable:

```
{  
  "update.positron.channel": "dailies"  
}
```

Los *daily builds* solo pasan pruebas automatizadas (no el QA completo), y se retienen 60 días. Las releases mensuales nunca se eliminan. No se recomienda para un entorno de trabajo principal si priorizas estabilidad.

1.7 6. Tour de la interfaz

1.7.1 Layout general (heredado de Code OSS)

```

Title Bar / Command Center
--

Activity Bar                      Editor Area
(Explorer,
  Git, Run,
  Extensions)                    --

                                Console / Terminal
                                --

Status Bar

```

A la derecha (o izquierda, según configures `workbench.sidebar.location`) se ubica la **barra lateral secundaria**, donde viven los paneles exclusivos de Positron.

1.7.2 Paneles exclusivos de Positron

Panel	Función
Console	Shell interactiva para Python (IPython/Ark) o R (IRKernel). Reemplaza al terminal para ejecución de código, con autocompletado e integración directa con Variables/Plots.
Variables Pane	Lista todas las variables del entorno activo: nombre, tipo, valor/tamaño, dimensiones (en dataframes). Doble clic en un dataframe lo abre en el Data Explorer.
Data Explorer	Visualización interactiva de dataframes: filtrado, ordenamiento, búsqueda, estadísticas por columna, exportación a CSV/Excel.
Plots Pane	Historial de gráficos generados, con zoom, pan y exportación a PNG/SVG/PDF.
Help Pane	Documentación contextual de funciones, paquetes y argumentos, con búsqueda integrada.
Connections Pane	Gestión de conexiones a bases de datos.
Packages Pane	Vista de paquetes R/Python instalados, adjuntos y desactualizados, con instalación/actualización integrada.

1.7.3 Console vs. Terminal — la distinción clave

	Terminal	Console
Qué es	Shell del sistema (zsh, bash, fish...)	Intérprete interactivo (IPython / IREKernel)

	Terminal	Console
Para qué sirve	Comandos del sistema, instalación de paquetes, git por CLI	Ejecutar código Python/R, inspeccionar variables, generar plots
Se integra con	Nada en particular	Variables Pane, Plots Pane, Data Explorer

1.8 7. Atajos de teclado esenciales

Positron hereda **todos** los atajos de VS Code y añade un conjunto propio para flujos de ciencia de datos.

1.8.1 Atajos específicos de Positron (los más importantes)

Atajo (Linux)	Acción
Ctrl+Enter	Ejecutar la línea/selección actual en la Console (si no hay selección, ejecuta la sentencia actual)
Ctrl+Alt+Home	Ejecutar desde el inicio del documento hasta la línea actual
Ctrl+Alt+End	Ejecutar desde la línea actual hasta el final del documento
Ctrl+Shift+O	Reiniciar el intérprete activo en la Console
Ctrl+Shift+S	Ejecutar el archivo completo en la Console (equivalente a <code>source()</code> en R o <code>%run</code> en Python)
F1	Mostrar ayuda contextual del símbolo bajo el cursor
Ctrl+K, F	Enfocar la Console
Ctrl+K, V	Enfocar el panel de Variables
Ctrl+L	Limpiar la Console

Nota: los atajos exactos pueden variar ligeramente según el teclado/distro. Si uno no coincide, ábrelo desde la Paleta de Comandos (Ctrl+Shift+P → *Preferences: Open Keyboard Shortcuts*) y búscalo por nombre de comando.

1.8.2 Atajos generales heredados de VS Code

Navegación y vistas:

Atajo	Acción
Ctrl+P	Quick Open (buscar archivo por nombre)
Ctrl+Shift+P	Paleta de comandos
Ctrl+,	Abrir Settings
Ctrl+B	Mostrar/ocultar barra lateral
Ctrl+J	Mostrar/ocultar panel inferior
Ctrl+Shift+E	Explorer
Ctrl+Shift+G	Source Control (Git)

Atajo	Acción
Ctrl+Shift+X	Extensions

Edición:

Atajo	Acción
Ctrl+Space	Disparar sugerencias de autocompletado
Alt+↑ / ↓	Mover línea arriba/abajo
Shift+Alt+↑ / ↓	Duplicar línea arriba/abajo
Ctrl+/	Comentar/descomentar línea
Ctrl+D	Seleccionar siguiente ocurrencia (multi-cursor)
Ctrl+Shift+L	Seleccionar todas las ocurrencias
Alt+Click	Añadir cursor manual

Navegación de código:

Atajo	Acción
Ctrl+G	Ir a línea
F12	Ir a definición
Shift+F12	Mostrar referencias
Ctrl+Shift+O	Ir a símbolo en el archivo

1.8.3 Crear tus propios atajos

Paleta de Comandos → Preferences: Open Keyboard Shortcuts (JSON)

Esto abre tu `keybindings.json` personal — ver ejemplos en la sección 15.

1.9 8. Extensiones: Open VSX y P3M**1.9.1 Por qué Positron no usa el VS Code Marketplace**

Por motivos de licencia (Microsoft no permite el acceso a su Marketplace fuera de builds oficiales de VS Code), Positron obtiene extensiones de un catálogo alternativo.

- **Por defecto**, Positron navega e instala extensiones desde **Posit Public Package Manager (P3M)**, infraestructura propia de Posit pensada para distribución confiable.
- El catálogo servido por P3M es **el mismo conjunto de extensiones disponible en Open VSX**, el proyecto open-source de Eclipse que actúa como alternativa neutral al Marketplace de Microsoft.
- Si prefieres consultar Open VSX directamente en vez de P3M, puedes cambiarlo con:

```
{
  "positron.extensions.gallerySource": "openvsx" // "p3m" (por defecto) | "openvsx"
}
```

Open VSX incluye la mayoría de extensiones populares de VS Code, pero no todas: algunos autores no publican ahí o no mantienen esa versión actualizada.

1.9.2 Extensiones incluidas por defecto

Built-in (núcleo, no editables): manejan funcionalidad esencial como los backends de R y Python. Se actualizan junto con Positron. Para verlas: busca @builtin en la pestaña de Extensions.

Bootstrapped (preinstaladas automáticamente, pero desinstalables):

Extensión	Función
charliermarsh.ruff	Linter y formateador de Python (escrito en Rust, muy rápido)
meta.pyrefly	Language server de Python: tipado, autocompletado, diagnósticos
posit.shiny	Soporte para apps Shiny (Python y R)
posit.publisher	Publicación a Posit Connect / Posit Connect Cloud
quarto.quarto	Soporte completo de Quarto (autocompletado YAML, render, preview)
posit.air-vscode	Soporte de R: formateo, sintaxis, debugging

Si no usas alguna (por ejemplo, R), puedes desactivarla sin desinstalarla:

```
{
  "extensions.allowed": {
    "posit.air-vscode": false,
    "*": true
  }
}
```

1.9.3 Extensiones NO compatibles — evitar instalarlas

Extensión	Por qué evitarla
REditorSupport	Conflicta directamente con posit.air-vscode, que ya cubre lo mismo de forma nativa
ms-python.python	Positron incluye un fork propio adaptado, con soporte nativo de consola y ayuda
GitHub.copilot	No funciona por restricciones de licencia de Microsoft fuera de VS Code oficial — usa Posit Assistant en su lugar

Si usas R y tienes lógica en tu .Rprofile pensada para el terminal de la extensión R de VS Code, debes excluirla explícitamente en Positron:

```
if (interactive() && Sys.getenv("RSTUDIO") == "" && Sys.getenv("POSITRON") == "") {
  ## código pensado solo para mejor comportamiento en terminal con VS Code + REditor
}
```

1.9.4 Confianza y seguridad de extensiones

Antes de instalar una extensión de terceros, Posit recomienda verificar:

- **Verified Publisher:** insignia que confirma que el autor controla ese *namespace* en Open VSX.

- **Descargas totales:** indicador de adopción.
- **Repositorio:** revisa el código fuente si es open source.
- **Calificaciones y fecha de la última publicación** (¿se mantiene activa?).

Positron te pedirá confirmar tu confianza la primera vez que instales una extensión de un *namespace* nuevo.

1.9.5 Extensiones recomendadas para instalar manualmente

Categoría	Extensión
Tema	Material Icon Theme, One Dark Pro, GitHub Theme
Formateo	Prettier (esbenp.prettier-vscode)
LaTeX	LaTeX Workshop (James-Yu.latex-workshop)
Markdown	Markdown All in One
Productividad	GitLens, Better Comments, Code Spell Checker (español)

1.9.6 Instalación manual (offline / .vsix)

Paleta de Comandos → Extensions: Install from VSIX...

1.10 9. Dónde vive la configuración

1.10.1 Ubicación del archivo `settings.json`

Linux: `~/.config/Positron/User/settings.json`
 macOS: `~/Library/Application Support/Positron/User/settings.json`
 Windows: `%APPDATA%\Positron\User\settings.json`

1.10.2 Cómo abrirlo

Vía interfaz gráfica (recomendado para explorar opciones):

File → Preferences → Settings (o Ctrl+,)

Directamente como JSON (recomendado para copiar/pegar configuraciones):

Paleta de Comandos → Preferences: Open User Settings (JSON)

1.10.3 Settings por workspace (proyecto específico)

Si quieres una configuración distinta solo para un proyecto puntual (por ejemplo, un repo Quarto que usa reglas de formato diferentes), crea:

`.vscode/settings.json`

dentro de la carpeta del proyecto. Estos valores tienen prioridad sobre tu configuración de usuario global, solo dentro de ese workspace.

1.11 10. Personalización visual (temas, iconos, fuentes)

1.11.1 Tema de color

Paleta de Comandos → Preferences: Color Theme (Ctrl+K Ctrl+T)

Temas incluidos por defecto: *Default Positron Dark*, *Default Positron Light*, y las variantes heredadas de Code OSS (Dark+, Light+). Para más opciones, instala temas desde la pestaña de Extensions (ej. One Dark Pro, Dracula, Nord, Gruvbox).

```
{
  "workbench.colorTheme": "Default Positron Dark"
}
```

1.11.2 Tema de iconos de archivos

Paleta de Comandos → Preferences: File Icon Theme

```
{
  "workbench.iconTheme": "material-icon-theme"
}
```

1.11.3 Personalizar colores específicos

```
{
  "workbench.colorCustomizations": {
    "[Default Positron Dark]": {
      "editor.background": "#1a1a1a",
      "sideBar.background": "#1e1e1e",
      "activityBar.background": "#252525"
    }
  }
}
```

1.11.4 Tipografía

Fuentes recomendadas con ligaduras (combinan símbolos como != o => en un solo glifo):

- JetBrains Mono
- Fira Code
- Cascadia Code
- Hack Nerd Font (recomendada para terminal, incluye íconos)

```
{
  "editor.fontFamily": "'JetBrains Mono', 'Fira Code', monospace",
  "editor.fontLigatures": true,
  "editor.fontSize": 13,
  "terminal.integrated.fontFamily": "'Hack Nerd Font', monospace",
  "terminal.integrated.fontSize": 12
}
```

1.12 11. Personalización del layout

1.12.1 Posición de la barra de actividad (íconos de Explorer, Git, etc.)

```
{
  "workbench.activityBar.location": "bottom" // "default" | "top" | "bottom" | "hidden"
}
```

1.12.2 Posición de la barra lateral principal

```
{
  "workbench.sideBar.location": "right" // "left" | "right"
}
```

1.12.3 Pestañas de editor

```
{
  "workbench.editor.showTabs": "single", // "multiple" | "single" | "none"
  "workbench.editor.tabSizing": "fixed" // "fit" | "shrink" | "fixed"
}
```

1.12.4 Barra de estado y breadcrumbs

```
{
  "workbench.statusBar.visible": false,
  "breadcrumbs.enabled": false
}
```

1.12.5 Modo Zen (máxima concentración, sin distracciones)

Paleta de Comandos → View: Toggle Zen Mode (Ctrl+K Z)

```
{
  "zenMode.centerLayout": true,
  "zenMode.hideTabs": true,
  "zenMode.hideStatusBar": true,
  "zenMode.hideActivityBar": true,
  "zenMode.restore": true
}
```

1.13 12. Configuración del editor

1.13.1 Lo esencial

```
{
  "editor.fontSize": 13,
  "editor.fontFamily": "'JetBrains Mono', monospace",
  "editor.lineNumbers": "on", // "on" | "off" | "relative" | "interval"
  "editor.rulers": [80, 160], // guías verticales de columna
  "editor.wordWrap": "off", // "off" | "on" | "wordWrapColumn" | "bounded"
  "editor.tabSize": 4,
  "editor.formatOnSave": true,
  "editor.formatOnPaste": true
}
```

1.13.2 Configuración por lenguaje (muy útil si trabajas Python + Quarto + LaTeX + R)

```
{
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff"
  },
  "[quarto]": {
    "editor.defaultFormatter": "quarto.quarto"
  },
  "[markdown]": {
    "editor.defaultFormatter": "esbenp.prettier-vscode",
    "editor.formatOnSave": true
  },
  "[latex]": {
    "editor.defaultFormatter": "James-Yu.latex-workshop"
  },
  "[r]": {
    "editor.defaultFormatter": "Posit.air-vscode",
    "editor.formatOnSave": true
  }
}
```

1.13.3 Python: intérprete y kernel

```
{
  "python.defaultInterpreterPath": "/ruta/a/tu/anaconda3/bin/python",
  "python.condaPath": "/ruta/a/tu/anaconda3/bin/conda",
  "python.terminal.activateEnvironment": true,
  "python.useBundledIpykernel": true,
  "python.enableAutoReload": true
}
```

1.13.4 R: formateo con Air

```
{
  "r.alwaysUseActiveTerminal": true,
  "r.bracketedPaste": true
}
```

El formateador `Posit.air-vscode` también admite un archivo `air.toml` en la raíz de un proyecto R, con opciones como `line-width`, `indent-width` e `indent-style`. Más en: <https://posit-dev.github.io/air/configuration.html>

1.13.5 Quarto

```
{
  "quarto.visualEditor.spellingDictionary": "es_ES"
}
```

Renderizar un documento:

Paleta de Comandos → Quarto: Render Document (o Ctrl+Shift+K)

```
quarto render documento.qmd
quarto render documento.qmd --to pdf
```

1.13.6 Terminal integrada

```
{
  "terminal.integrated.defaultProfile.linux": "zsh",
  "terminal.integrated.inheritEnv": false,
  "terminal.integrated.scrollback": 5000,
  "terminal.integrated.copyOnSelection": true
}
```

1.14 13. Configuración de Git

1.14.1 Configuración inicial de Git (una sola vez, fuera de Positron)

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"
```

1.14.2 Settings recomendados de Git en Positron

```
{
  "git.enableSmartCommit": true,
  "git.autofetch": true,
  "git.confirmSync": false,
  "git.openRepositoryInParentFolders": "always",
  "git.defaultBranchName": "main"
}
```

1.14.3 Panel de Source Control

Ctrl+Shift+G

Desde ahí puedes: ver cambios, hacer *stage/unstage*, escribir el mensaje de commit, hacer *push/pull*, crear ramas y ver el historial — todo sin salir del editor.

1.15 14. Posit Assistant (IA integrada)

Cambio importante de nomenclatura (versión 2026.06.0): el antiguo *Positron Assistant* fue renombrado a **Posit Assistant**. El setting `positron.assistant.enable` quedó **deprecado** en favor de `assistant.enabled`. El antiguo seguirá funcionando un tiempo por compatibilidad, pero conviene migrar ya.

1.15.1 Activarlo

```
{
  "assistant.enabled": true,
  "positron.assistant.alwaysEnableApplyInEditorAction": true,
  "positron.assistant.showTokenUsage.enable": true
}
```

1.15.2 Usarlo

Paleta de Comandos → Positron: Open Assistant

Funciones principales: generar código, explicar código existente, refactorizar, documentar y ayudar a depurar — con acceso directo a tu sesión activa de R o Python.

1.15.3 Databot

Asistente especializado en análisis de datos conversacional, también accesible desde la paleta de comandos:

Paleta de Comandos → Positron: Open Databot

Ejemplos de instrucciones útiles: “*crea un gráfico de dispersión de x vs y*”, “*agrupa por categoría y calcula la media*”, “*filtra datos donde columna > 100*”.

1.16 15. Atajos personalizados (keybindings.json)

Como Positron está construido sobre VS Code, puedes definir atajos propios para cualquier comando, incluidos los específicos de Positron (`workbench.action.executeCode.console`, etc.).

Paleta de Comandos → Preferences: Open Keyboard Shortcuts (JSON)

1.16.1 Ejemplo: abrir el Data Explorer con F2 desde el panel de Variables

```
[
  {
    "key": "f2",
    "command": "positron.dataExplorer.openDataExplorer",
    "when": "positronVariablesFocus"
  }
]
```

1.16.2 Ejemplo: generar un reprex en R desde tu selección

```
[
  {
    "key": "ctrl+shift+r",
    "command": "workbench.action.executeCode.console",
    "when": "editorTextFocus",
    "args": {
      "langId": "r",
      "code": "reprex::reprex_selection()",
      "focus": true
    }
  }
]
```

El terminal integrado puede interceptar algunos atajos antes de que lleguen a Positron. Si un atajo no responde dentro del terminal, agrégalo a `terminal.integrated.commandsToSkipShell`:

```
{
  "terminal.integrated.commandsToSkipShell": [
    "workbench.action.positronConsole.focusConsole"
  ]
}
```

1.17 16. Solución de problemas comunes

1.17.1 El intérprete de Python no se detecta

Paleta de Comandos → Python: Select Interpreter

O fíjalo manualmente:

```
{
  "python.defaultInterpreterPath": "/ruta/a/tu/python"
}
```

1.17.2 R no funciona

```
which R
R --version
```

```
pak::pak(c("usethis", "cli", "crayon", "rlang"))
```

1.17.3 La Console no inicia

Ctrl+Shift+0 # reiniciar el intérprete

```
pip install ipython
```

1.17.4 Las extensiones esperadas no aparecen

Recuerda: Positron usa P3M/Open VSX, no el VS Code Marketplace. Si una extensión no está disponible ahí, instálala manualmente desde un archivo .vsix (ver sección 8).

1.17.5 La terminal no hereda el PATH correctamente

```
{
  "terminal.integrated.inheritEnv": false
}
```

Y añade la ruta necesaria directamente en tu shell config (~/.zshrc):

```
export PATH="/ruta/a/tu/anaconda3/bin:$PATH"
```

1.17.6 Los plots no aparecen (Python)

```
import matplotlib.pyplot as plt
plt.ion() # modo interactivo
```

1.17.7 Quarto no renderiza

```
quarto check
```

Paleta de Comandos → Quarto: extensión → Reload

1.17.8 Los settings no se guardan

Verifica la ruta y permisos del archivo:

```
ls -la ~/.config/Positron/User/settings.json
chmod 644 ~/.config/Positron/User/settings.json
```

1.17.9 Rendimiento lento

```
{
  "editor.minimap.enabled": false,
  "terminal.integrated.scrollback": 1000
}
```

Y revisa si tienes extensiones innecesarias activas que puedas desactivar desde la pestaña de Extensions.

1.18 17. Mi configuración final (settings.json)

Configuración lista para copiar y pegar completa en:

```
~/.config/Positron/User/settings.json
```

Está organizada por bloques temáticos, comentada en español, y usa nombres de settings **actualizados a la versión 2026.06.0** (por ejemplo: assistant.enabled en vez del deprecado positron.assistant.enable; packages.enabled en vez del deprecado positron.packages.enable).

Antes de pegar: revisa especialmente la sección 1 (rutas de Python/Anaconda) y ajusta tu-usuario a tu ruta real. El resto puedes dejarlo tal cual y refinar sobre la marcha.

```
//
// CONFIGURACIÓN DE POSITRON - Edison Achalma (UNSCH, Ayacucho)
// Positron 2026.06.0 · Python (Anaconda) + Quarto + LaTeX + R + zsh + Linux
//
{
  //
  // 1. PYTHON (ANACONDA / CONDA)
  //
  "python.defaultInterpreterPath": "/home/tu-usuario/anaconda3/bin/python",
  "python.condaPath": "/home/tu-usuario/anaconda3/bin/conda",
  "python.terminal.activateEnvironment": true,
  "python.terminal.activateEnvInCurrentTerminal": true,
  "python.useBundledIpykernel": true,
  "python.enableAutoReload": true,
  "interpreters.startupBehavior": "always",
  "interpreters.discoveryCache.enabled": true,

  //
  // 2. PAQUETES (panel "Packages")
  //
  "packages.enabled": true,
  "packages.r.installer": "auto",

  //
```

```
// 3. TERMINAL
//
"terminal.integrated.inheritEnv": false,
"terminal.integrated.fontSize": 12,
"terminal.integrated.fontFamily": "'Hack Nerd Font', 'FiraCode Nerd Font', monospace",
"terminal.integrated.copyOnSelection": true,
"terminal.integrated.cursorBlinking": true,
"terminal.integrated.cursorStyle": "underline",
"terminal.integrated.defaultProfile.linux": "zsh",
"terminal.integrated.enablePersistentSessions": false,
"terminal.integrated.scrollback": 5000,

//
// 4. EDITOR
//
"editor.fontSize": 13,
"editor.fontFamily": "'JetBrains Mono', 'Fira Code', 'Hack Nerd Font', monospace",
"editor.fontLigatures": true,
"editor.lineNumbers": "on",
"editor.rulers": [80, 160],
"editor.wordWrap": "off",
"editor.minimap.enabled": true,
"editor.minimap.autohide": true,
"editor.scrollbar.vertical": "hidden",
"editor.scrollbar.horizontal": "hidden",
"editor.cursorBlinking": "smooth",
"editor.cursorSmoothCaretAnimation": "on",
"editor.cursorStyle": "line",
"editor.formatOnSave": true,
"editor.formatOnPaste": true,
"editor.tabSize": 4,
"editor.insertSpaces": true,
"editor.detectIndentation": true,
"editor.renderWhitespace": "selection",
"editor.glyphMargin": false,
"editor.bracketPairColorization.enabled": true,
"editor.guides.bracketPairs": true,
"editor.guides.indentation": false,
"editor.matchBrackets": "never",
"editor.linkedEditing": true,
"editor.suggest.preview": true,
"editor.inlineSuggest.enabled": true,
"editor.inlineSuggest.syntaxHighlightingEnabled": true,
"editor.quickSuggestions": {
  "other": true,
  "comments": false,
  "strings": false
}
```

```
},

//
// 5. WORKBENCH (apariencia general del IDE)
//
"workbench.colorTheme": "Default Positron Dark",
"workbench.iconTheme": "material-icon-theme",
"workbench.activityBar.location": "bottom",
"workbench.sideBar.location": "right",
"workbench.statusBar.visible": false,
"workbench.editor.showTabs": "single",
"workbench.editor.tabSizing": "fixed",
"workbench.editor.highlightModifiedTabs": true,
"workbench.editor.empty.hint": "hidden",
"window.commandCenter": true,
"breadcrumbs.enabled": false,

//
// 6. FILES
//
"files.autoSave": "onFocusChange",
"files.associations": {
  "renv.lock": "json",
  "*.qmd": "quarto",
  "*.Rmd": "rmarkdown",
  "*.csv": "csv"
},
"files.exclude": {
  "**/_site": true,
  "**/.quarto": true,
  "**/__pycache__": true,
  "**/.ipynb_checkpoints": true
},

//
// 7. GIT
//
"git.enableSmartCommit": true,
"git.autofetch": true,
"git.confirmSync": false,
"git.openRepositoryInParentFolders": "always",
"git.defaultBranchName": "main",
"git.branchProtection": ["main"],
"git.pullTags": true,
"git.requireGitUserConfig": true,
"git.useForcePushWithLease": true,
"git.useForcePushIfIncludes": true,
```

```
//
// 8. CONFIGURACIÓN POR LENGUAJE
//
"[python]": {
  "editor.defaultFormatter": "charliermarsh.ruff"
},
"[quarto]": {
  "editor.defaultFormatter": "quarto.quarto"
},
"[markdown]": {
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.formatOnSave": true
},
"[yaml]": {
  "editor.defaultFormatter": "quarto.quarto"
},
"[latex]": {
  "editor.defaultFormatter": "James-Yu.latex-workshop"
},
"[r]": {
  "editor.defaultFormatter": "Posit.air-vscode",
  "editor.formatOnSave": true
},
"[jsonc]": {
  "editor.defaultFormatter": "vscode.json-language-features"
},

//
// 9. NOTEBOOKS (Jupyter / .ipynb)
//
"notebook.output.wordWrap": true,
"jupyter.interactiveWindow.textEditor.executeSelection": true,

//
// 10. QUARTO
//
"quarto.visualEditor.spellingDictionary": "es_ES",
"yaml.schemas": {
  "https://quarto.org/schemas/quarto.yaml": "*.qmd"
},

//
// 11. POSIT ASSISTANT (antes "Positron Assistant")
//
"assistant.enabled": true,
"positron.assistant.alwaysEnableApplyInEditorAction": true,
"positron.assistant.showTokenUsage.enable": true,
```

```
//  
// 12. MATERIAL ICON THEME  
//  
"material-icon-theme.files.color": "#42a5f5",  
"material-icon-theme.folders.color": "#ef5350",  
"material-icon-theme.folders.theme": "specific",  
"material-icon-theme.opacity": 1,  
"material-icon-theme.saturation": 1,  
  
//  
// 13. PRETTIER  
//  
"prettier.printWidth": 80,  
"prettier.proseWrap": "preserve",  
"prettier.requirePragma": false,  
"prettier.requireConfig": false,  
  
//  
// 14. LATEX WORKSHOP  
//  
"latex-workshop.formatting.latex": "latexindent",  
"latex-workshop.latex.option.maxPrintLine.enabled": false,  
  
//  
// 15. ACTUALIZACIONES  
//  
"update.mode": "default"  
}
```

-
- Instalación: <https://positron.posit.co/install.html>
 - Descargas y notas de versión: <https://positron.posit.co/download.html>
 - Extensiones: <https://positron.posit.co/extensions.html>
 - Atajos de teclado: <https://positron.posit.co/keyboard-shortcuts.html>
 - Lanzar desde terminal: <https://positron.posit.co/add-to-path.html>
 - Actualizaciones: <https://positron.posit.co/updates.html>
 - Repositorio oficial: <https://github.com/posit-dev/positron>

2 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1.  [Comandos De Informacion Windows](#)
2.  [Adb](#)
3.  [Limpieza Y Optimizacion De Pc](#)
4.  [Screpy](#)

5.  [Gestionar Versiones De Jdk En Kubuntu](#)
6.  [Instalar Tor Browser](#)
7.  [Crear Enlaces Duros O Hard Link En Linux](#)
8.  [Comandos Vim](#)
9.  [Guia De Git Y Github](#)
10.  [00 Primeros Pasos En Linux](#)
11.  [02 Distribuciones Linux](#)
12.  [03 Instalacion Linux](#)
13.  [04 Administracion Particiones Volumenes](#)
14.  [Atajos De Teclado Y Comandos Para Usar Vim](#)
15.  [Instalando Specitify](#)
16.  [Gestiona Tus Dotfiles Con Gnu Stow](#)
17.  [Guia De Ranger](#)
18.  [Guia De Kitty Terminal](#)
19.  [Guia De Positron Ide](#)

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!